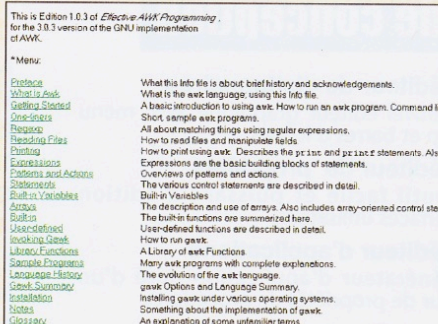


AWK

Jean-Marc Quéré
apiend@club-internet.fr

Les manipulations quotidiennes de fichiers requièrent souvent la programmation de «moulinettes» à usage unique. Les langages de développement nécessitent de nombreuses lignes de code pour ouvrir les flux d'entrée et de sortie, interpréter les données collectées et assurer leur mise en forme. De plus, de tels environnements – en général volumineux – doivent être préalablement installés. L'idéal serait de disposer d'un langage de script, simple à mettre en œuvre, compact et orienté vers la recherche et l'analyse de séquences et d'expressions.

Cette problématique a intéressé des esprits particulièrement brillants. Ainsi Alfred V. Aho (expert ès-compilation), Peter J. Weinberger (factorisation des polynômes, ...), et Brian W. Kernighan (à qui l'on doit également le C) se sont penchés sur sa résolution par l'implémentation d'un nouveau langage. Désignée par l'association des initiales de ses concepteurs, la première version d'AWK – sous la forme d'un interpréteur – a donc été finalisée en 1977. L'usage aidant, les spécificités du langage évoluèrent à un rythme soutenu. Les versions se succédèrent ainsi jusqu'en



➤ L'aide de GAWK (une référence).

1985. Date à laquelle, l'état de l'art fût universellement reconnu comme une norme. Les améliorations les plus significatives portaient alors sur la possibilité de créer des fonctions utilisateurs, sur la gestion de flux multiples et sur la prise en charge des expressions régulières. Largement diffusé dans le monde UNIX (intégré au distribution du System V), AWK acquit ses lettres de noblesses lors de l'intégration de ses spécificités, à la rubrique «langage de commandes et utilitaires» du standard POSIX (en l'occurrence 1003.2). À l'initiative de Richard Stallman (GNU) une implémentation libre d'AWK (également sous forme interprétée) apparut dès 1986. Elle évolua et continue

```
06/11/2000 09:15 <DIR> bin
06/11/2000 09:21 111 ex000.awk
06/11/2000 09:58 10 ex000.txt
06/11/2000 09:57 101 ex001.awk
06/11/2000 10:21 492 240 gawk-3.0.3.zip
06/11/2000 09:16 <DIR> hlp
06/11/2000 09:15 <DIR> share
5 fichier(s) 629 166 octets
5 Rép(s) 1 444 786 176 octets libres

C:\gawk>dir | awk '{if ($1 ~ "ex000") print $1}'
06/11/2000 09:21 111 ex000.awk
06/11/2000 09:58 10 ex000.txt
15/02/1998 10:21 492 240 gawk-3.0.3.zip
06/11/2000 09:16 <DIR> hlp
06/11/2000 09:15 <DIR> share
```

➤ AWK en action.

d'évoluer, au gré de ses contributeurs, par l'adjonction d'améliorations et des nécessaires mises au point. GAWK est disponible pour de nombreux systèmes d'exploitation à partir du site du GNU (<http://www.gnu.org>). Des variantes commerciales sont également diffusées, en particulier TAWK (<http://www.tasoft.com>). Particulièrement complète, la distribution constitue une véritable chaîne de développement (en lignes de commandes) permettant la génération d'exécutables natifs Dos, OS/2, Solaris et Windows (avec l'exploitation des sockets et des dll).

En pratique

La réalisation d'un programme AWK s'effectue «à la main», à l'aide d'un éditeur de texte : «notepad» pour Windows ou «vi» pour UNIX conviennent parfaitement. Trois blocs fonctionnels sont disponibles (mais non obligatoires) : «BEGIN», «expression régulière» et «END». Les instructions des blocs «BEGIN» et «END» s'exécutent respectivement au début et à la fin du programme, celles rattachées à une expression régulière sont exécutées lorsque les conditions requises par celle-ci sont vérifiées.

ex000.awk

```
BEGIN {
    print "DEBUT"
    CPT=0
}
/A/ {
    CPT++
}
END {
    print "NB DE LIGNE(S) AVEC UN 'A' : " CPT
}
```

L'exemple ci-dessus compte le nombre de lignes avec un «A». Son activation s'effectue à la ligne de commande à l'aide de la commande «awk -f ex000.awk». Le message «DEBUT» s'affiche

GNU's Not Unix!



[Allemand | Anglais | Catalan | Coréen | Espagnol | Français | Hongrois | Italien | Japonais | Néerlandais | Portugais | Turc]

Bienvenue sur le serveur web du Projet GNU, www.gnu.org. Le Projet GNU a été lancé en 1984 afin de développer un système d'exploitation complet et semblable à Unix qui soit un logiciel libre : le système GNU. Des variantes du système d'exploitation GNU, utilisant le noyau «Linux», sont utilisées largement à présent; bien que ces systèmes soient communément nommés par le terme «Linux», ils sont plus exactement appelés «systèmes GNU/Linux».

- Ce que nous offrons
- Notre raison d'être
- Vers où nous allons
- Aidez-nous à y arriver
- Qui nous sommes
- Témoignages d'utilisateurs de logiciels GNU

[Logiciels](#) | [Projets](#) | [Agenda](#) | [Passer commande](#) | [Documentation](#)

[Possibilités d'emploi](#) | [Faites-vous aider](#) | [L'an 2000](#) | [La reconnaissance de GNU](#) | [Brave GNU World](#)

➤ La version GNU (libre, interpréteur) d'AWK.

sur la console. Vous pouvez alors saisir quelques lignes de texte, séparées par un retour-chariot, puis pour terminer les traitements frappez [CTRL]-Z (touche F6 sous Windows), puis ENTRÉE. L'intérêt d'AWK étant la manipulation de fichiers, la commande peut être complétée par le nom des fichiers concernés.

ex000.txt

A
B
BA

La commande «**awk -f ex000.awk ex000.txt**» retourne «**DEBUT**», puis «**NB DE LIGNE(S) AVEC UN 'A' : 2**». La liste des fichiers à traiter peut comporter des «jokers», par exemple : «***.bat**».

Les expressions régulières («**/A/**» dans notre exemple) permettent de limiter l'exécution des instructions concernées aux seules lignes compatibles : comprenant la séquence décrite. Les principales règles d'écriture sont les suivantes : «**.**» (le point) correspond à un caractère quelconque, «**[ABC]**» liste les caractères (A ou B ou C), «**[A-Z]**» définit l'ensemble des caractères de A à Z, «*****» «**+**» «**?**» indiquent respectivement une répétition de 0 à n, 1 à n, 0 à 1. La description peut être relative au début de la ligne (préfixée par «**^**») ou à la fin (postfixée par «**\$**»).

ex001.awk

```
/BA*/ { printf «1» }
/A+ / { printf «2» }
/B? / { printf «3» }
/B[AB] / { printf «4» }
// { print «» }
```

La commande «**awk -f ex001.awk ex000.txt**» affiche «**23**», «**13**», «**1234**». L'expression «**//**» (qui peut être omise) est vérifiée pour toutes les lignes. Elle est utilisée ici pour sauter une ligne. L'autre intérêt d'AWK consiste en sa faculté de découper la ligne lue (notée \$0) en champs (\$1 à \$n) à partir d'un séparateur de champs (par défaut l'espace) et de ligne (par défaut le retour-chariot).

Windows 2000 seulement

```
dir | awk "/:/ { print $1 \" \\\" $4 }»
```

La commande ci-dessus récupère le flux (pipe) issu de la commande «**dir**»



Thompson Automation Software

Comparison of TAWK with the C Programming Language

Here is a comparison of the TAWK language to the "C" language. These example programs were written using both a "C" compiler and the TAWK Compiler, then the development and execution times of the two languages were compared. These are real results. We didn't make this up.

- [Home Page](#)
- [Thompson Toolkit](#)
- [TAWK](#)
- [Patches & Downloads](#)
- [Prices & Ordering](#)

Program 1:
Description: Search files for pattern.
TAWK Compiler: Development Time: 1 minute; Execution Time: 4.9 sec;
C Compiler: Development Time: 15 minutes; Execution Time: 4.4 sec;

Program 2:
Description: Convert text file to postscript.
TAWK Compiler: Development Time: 15 minutes; Execution Time: 15.2 sec;
C Compiler: Development Time: 90 minutes; Execution Time: 17.9 sec;

Program 3:
Description: Typesetter program.
TAWK Compiler: Development Time: 8 hours; Execution Time: 6.3 sec;
C Compiler: Development Time: 4 days; Execution Time: 2.3 sec;

Notes: C programs were compiled using Microsoft C, large model, with maximum optimization (/Ox option).

► La version - la plus aboutie - d'AWK (commerciale, interpréteur et compilateur).

et affiche la date et le nom de chaque fichier (ou répertoire). L'espace est l'opérateur de concaténation. L'autre exemple, adapté au «**dir**» de 95 permet d'extraire les répertoires.

Windows 9x seulement

```
dir | awk "/<REP>/ { print $1 }
```

Les variables d'AWK ne sont pas typées. Leur nature est déterminée par les opérateurs employés. «**BEGIN { A="12"; B=35; print A B; print A+B }**» affiche «**1235**», puis «**47**». L'opérateur de concaténation provoque la conversion de B en chaîne. L'addition induit l'interprétation de la valeur de la chaîne A. Les variables peuvent également être exploitées sous forme de tableau sans déclaration préalable. Les indices sont au choix des valeurs numériques ou alphanumériques (voire les deux). La structure «**for**» dispose même d'une déclinaison spécifique aux tableaux «**for (variant in tableau)**» permettant leur parcours.

ex003.awk

```
BEGIN {
  split("CECI EST UN TEST",tab," ")
  for (i in tab) {
    print i,tab[i]
    idx[tab[i]]=i
  }
  for (i in idx) {
    print i,idx[i]
  }
}
```

tions orientées traitement de données. L'une des plus célèbres est le «**split**» qui restitue sous la forme d'un tableau (**tab**) une chaîne («**CECI ...**») suite à son découpage à l'aide du séparateur indiqué (espace par défaut).

L'absence de typage des variables permet la rédaction de fonction générique. «**max**» retourne la valeur la plus grande quelle que soit la nature des valeurs. Avec un langage typé, deux fonctions auraient dû être rédigées, par contre, l'absence de typage peut produire des résultats assez inattendus, exemple : **max(100, "34")** retourne ... **34** ! Lorsqu'un opérateur de comparaison implique une chaîne, l'ensemble des valeurs comparées sont converties en chaîne et «**100**» est plus petit que «**34**» («**1**» précède «**3**» dans la table ascii).

ex004.awk

```
function max(v1,v2) {
  return v1>v2 ? v1 : v2;
}
BEGIN {
  print max("12","34");
  print max(100,34);
}
```

Révolutionnaire

Novateur, AWK introduit deux notions spécifiques : les expressions régulières et leur gestion événementielle. À l'état «**pur**» (comprendre originel non dérivé), il n'est exploité qu'en complément des interprètes de commandes des différents systèmes d'exploitation. ■

AWK comporte de nombreuses instruc-